MSDM 5001
Introduction to Computational and Modeling Tools

Lecture 3: Relational Database and SQL

Before we start...

- Make sure you have SQLite installed on your system.
- Verify by running “`sqlite3 --version`”
- If you don’t have it on your system, just run “`sudo <package_manager> install sqlite`”

Relational data

- AKA tabular data
- Rows: records
- Columns: attributes/fields
- Examples:
  - Students in a class
  - Stocks
  - Experimental measurements

Date	Open	High	Low	Close	Volume
1993-01-29 00:00:00-05:00	24.397778	24.397778	24.276396	24.380438	1003200
1993-02-01 00:00:00-05:00	24.397765	24.553827	24.397765	24.553827	480500
1993-02-02 00:00:00-05:00	24.536510	24.623211	24.484489	24.605871	201300
1993-02-03 00:00:00-05:00	24.640545	24.883309	24.623205	24.865969	529400
1993-02-04 00:00:00-05:00	24.952680	25.022041	24.675235	24.970020	531500
...	...	...	...	...	...
2025-09-09 00:00:00-04:00	648.969971	650.859985	647.219971	650.330017	66133900
2025-09-10 00:00:00-04:00	653.619995	654.549988	650.630005	652.210022	78034500
2025-09-11 00:00:00-04:00	654.179993	658.330017	653.590027	657.630005	69934400
2025-09-12 00:00:00-04:00	657.599976	659.109985	656.900024	657.409973	72780100
2025-09-15 00:00:00-04:00	659.640015	661.039978	659.340027	660.909973	63652300

Relational Database

- A way to store tabular data
- Question:
  Why not just use raw text files (e.g. csv), and then use pandas/polars/excel?
  - Data validity
    - What if some integer fields have been entered as strings?
    - What if we have multiple related tables to manage?
  - Performance (see later)
    - What if the file is larger than our memory size? (Not uncommon!)
    - What if we want to allow multiple users to access the data concurrently?

Keys

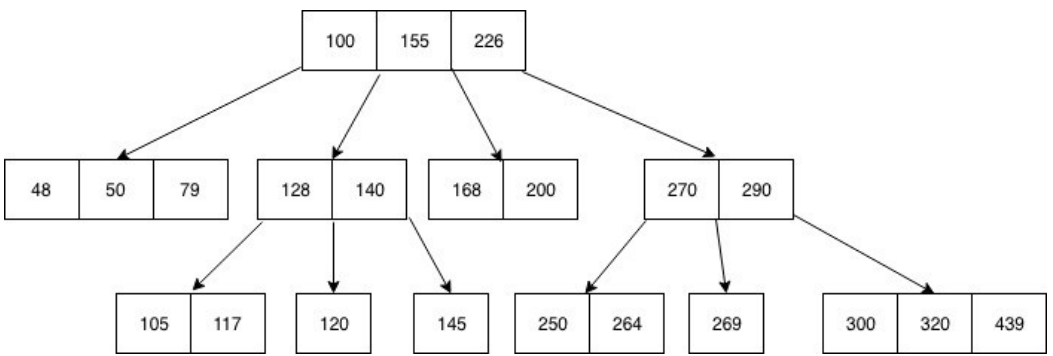
- Columns that can uniquely identify different rows
- Declared when creating a database
- Two main types:
  - Primary keys
  - Foreign keys – primary keys in another table
- Examples:
  - “Students” table with fields: name, student ID, age, courses registered
  - “Books” table with fields: title, authors, price

How a database process queries

- Common operations:
  read and write a large block of data depending on column values
- Naive implementation: plain text csv
  - Insert: O(1)
  - Lookup: O(n)
  - Delete: O(n)
- How can we make these operations faster?

B-trees ***

- Natural extension of binary tree:
  - Balanced tree data structure
  - Each node contains between m/2 and m children (m is called the order) (except root and leave nodes)
  - A non-leaf node with k children contains k−1 keys



Insert: ?

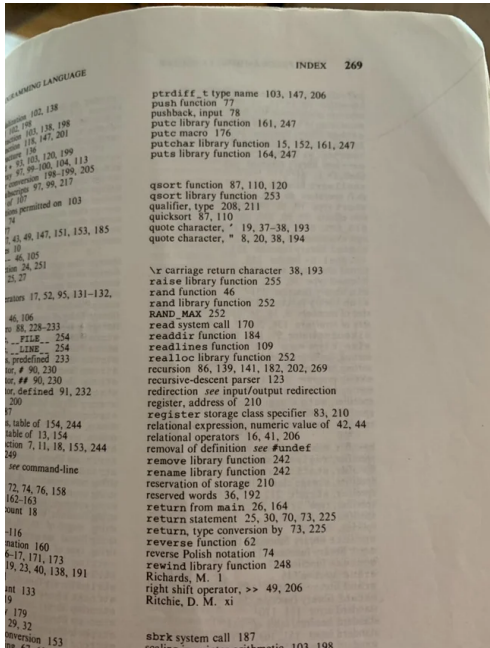
Delete: ?

Lookup: ?

*** Real databases use something called B+ trees

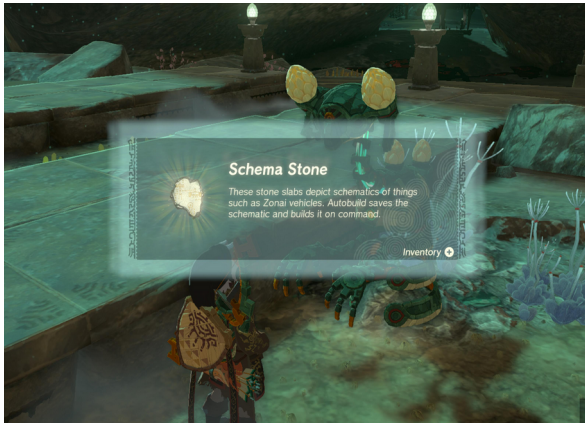
Index

- A column is “indexed” if it is used to construct a B-tree
- Increases lookup speed if the column is used a lot
- Primary key is often (but not always) indexed
- Analogy: index pages in a book
- Question: Why not just index all columns?



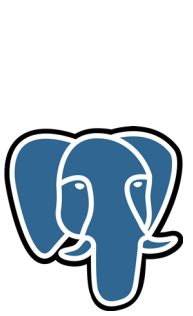
Jargon: “Schema”

- Formal description of entire database
  - What tables are in the database
  - Relationships between tables (e.g. foreign keys)
  - What fields go in each table, and their types
  - What are the keys in each table



RDBMS

- Different implementations of various database concepts
- Different functionalities
- Examples:
  - MySQL
  - PostgresSQL
  - SQLite
  - Oracle Database
  - Microsoft SQL server
  - ...



SQLite

- Start a connection by running `sqlite3 <database_name.db>`
- Now you enter a shell-like environment, you can enter various commands
  - `.schema`: check schema of the database
  - `.mode`: configures how output is formatted (csv, list, column, …)
  - `.output`: configures where the output is directed (default stdout)
  - `.headers [on]/[off]`: configures if headers are displayed
  - `.tables`: list all tables
  - `.databases`: see what databases are attached to current session
  - `<SQL_commands>;` : execute SQL statements (see next)

Structured Query Language (SQL)

- Most basic commands: `SELECT` and `WHERE`

```sql
SELECT [DISTINCT] column1, column2, ...
FROM table_name
WHERE condition
```

ID	First Name	Last Name	Department	Salary	Hire Year	Job Level
1	John	Doe	Sales	$55,000	2022	Senior
2	Jane	Smith	IT	$65,000	2021	Senior
3	Mike	Johnson	Sales	$52,000	2023	Junior
4	Sarah	Wilson	HR	$48,000	2022	Senior
5	David	Brown	IT	$72,000	2020	Senior
6	Lisa	Davis	Marketing	$59,000	2023	Senior
7	Tom	Miller	Sales	$58,000	2021	Senior
8	Amy	Garcia	IT	$68,000	2022	Senior
9	Chris	Lee	Sales	$45,000	2023	Junior
10	Emma	Taylor	IT	$63,000	2023	Junior
11	Alex	Moore	Marketing	$51,000	2022	Junior
12	Ryan	White	HR	$46,000	2023	Junior

```sql
-- Select all columns from employees table
SELECT * FROM employees

-- Select specific columns
SELECT first_name, last_name, salary FROM employees

-- Filter results with WHERE clause
SELECT first_name, last_name
FROM employees
WHERE department = 'Sales'

-- Multiple conditions
SELECT first_name, last_name
FROM employees
WHERE department = 'Sales' AND salary > 50000
```

Conditions in WHERE clause

- Pattern matching: `LIKE`

ID	First Name	Last Name	Department	Salary	Hire Year	Job Level
1	John	Doe	Sales	$55,000	2022	Senior
2	Jane	Smith	IT	$65,000	2021	Senior
3	Mike	Johnson	Sales	$52,000	2023	Junior
4	Sarah	Wilson	HR	$48,000	2022	Senior
5	David	Brown	IT	$72,000	2020	Senior
6	Lisa	Davis	Marketing	$59,000	2023	Senior
7	Tom	Miller	Sales	$58,000	2021	Senior
8	Amy	Garcia	IT	$68,000	2022	Senior
9	Chris	Lee	Sales	$45,000	2023	Junior
10	Emma	Taylor	IT	$63,000	2023	Junior
11	Alex	Moore	Marketing	$51,000	2022	Junior
12	Ryan	White	HR	$46,000	2023	Junior

```sql
-- Names starting with 'J'
SELECT * FROM employees WHERE first_name LIKE 'J%'

-- Names ending with 'son'
SELECT * FROM employees WHERE last_name LIKE '%son'

-- Names containing 'an'
SELECT * FROM employees WHERE first_name LIKE '%an%'

-- Exactly 4 characters (underscore represents single character)
SELECT * FROM products WHERE product_code LIKE '____'

-- Second character is 'a'
SELECT * FROM employees WHERE first_name LIKE '_a%'
```

Conditions in WHERE clause

- Membership filtering: `IN` and `BETWEEN`

```sql
-- Multiple specific values
SELECT * FROM employees WHERE department [NOT] IN ('Sales', 'Marketing', 'IT')

-- Salary between a range
SELECT * FROM employees WHERE salary between 50000 AND 60000
```

ID	First Name	Last Name	Department	Salary	Hire Year	Job Level
1	John	Doe	Sales	$55,000	2022	Senior
2	Jane	Smith	IT	$65,000	2021	Senior
3	Mike	Johnson	Sales	$52,000	2023	Junior
4	Sarah	Wilson	HR	$48,000	2022	Senior
5	David	Brown	IT	$72,000	2020	Senior
6	Lisa	Davis	Marketing	$59,000	2023	Senior
7	Tom	Miller	Sales	$58,000	2021	Senior
8	Amy	Garcia	IT	$68,000	2022	Senior
9	Chris	Lee	Sales	$45,000	2023	Junior
10	Emma	Taylor	IT	$63,000	2023	Junior
11	Alex	Moore	Marketing	$51,000	2022	Junior
12	Ryan	White	HR	$46,000	2023	Junior

Sorting

- `ORDER BY`

ID	First Name	Last Name	Department	Salary	Hire Year	Job Level
1	John	Doe	Sales	$55,000	2022	Senior
2	Jane	Smith	IT	$65,000	2021	Senior
3	Mike	Johnson	Sales	$52,000	2023	Junior
4	Sarah	Wilson	HR	$48,000	2022	Senior
5	David	Brown	IT	$72,000	2020	Senior
6	Lisa	Davis	Marketing	$59,000	2023	Senior
7	Tom	Miller	Sales	$58,000	2021	Senior
8	Amy	Garcia	IT	$68,000	2022	Senior
9	Chris	Lee	Sales	$45,000	2023	Junior
10	Emma	Taylor	IT	$63,000	2023	Junior
11	Alex	Moore	Marketing	$51,000	2022	Junior
12	Ryan	White	HR	$46,000	2023	Junior

```sql
-- Sort by single column (ascending by default)
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary

-- Sort by multiple columns
SELECT first_name, last_name, department, salary
FROM employees
ORDER BY department ASC, salary DESC
```

Aggregate functions

- SQL also supports computing (some) aggregates

```sql
-- Count all rows
SELECT COUNT(*) FROM employees

-- Count non-NULL values in a column
SELECT COUNT(phone_number) FROM employees

-- Count distinct values
SELECT COUNT(DISTINCT department) FROM employees

-- Total salary expenditure
SELECT SUM(salary) FROM employees

-- Average salary
SELECT AVG(salary) FROM employees

-- Highest and lowest salary
SELECT MAX(salary), MIN(salary) FROM employees
```

Can also give names:

```sql
SELECT
    COUNT(*) AS total_employees,
    AVG(salary) AS average_salary,
    MAX(salary) AS highest_salary,
    MIN(salary) AS lowest_salary,
    SUM(salary) AS total_payroll
FROM employees
```

What would happen if I execute this?

```sql
SELECT COUNT(*), salary FROM employees
```

Window functions

- ROW_NUMBER(), RANK(), DENSE_RANK()

```
FUNCTION_NAME() OVER (
    PARTITION BY column     -- Optional: creates groups
    ORDER BY column         -- Required: defines ranking order
)
```

ID	First Name	Last Name	Department	Salary	Hire Year	Job Level
1	John	Doe	Sales	$55,000	2022	Senior
2	Jane	Smith	IT	$65,000	2021	Senior
3	Mike	Johnson	Sales	$52,000	2023	Junior
4	Sarah	Wilson	HR	$48,000	2022	Senior
5	David	Brown	IT	$72,000	2020	Senior
6	Lisa	Davis	Marketing	$59,000	2023	Senior
7	Tom	Miller	Sales	$58,000	2021	Senior
8	Amy	Garcia	IT	$68,000	2022	Senior
9	Chris	Lee	Sales	$45,000	2023	Junior
10	Emma	Taylor	IT	$63,000	2023	Junior
11	Alex	Moore	Marketing	$51,000	2022	Junior
12	Ryan	White	HR	$46,000	2023	Junior

```
-- Rank salary by department
-- Same rank for ties, skip next rank
-- Can use ROW_NUMBER() or DENSE_RANK()
SELECT
        first_name,
        last_name,
        RANK() OVER (
                PARTITION BY Department
                ORDER BY Salary ASC
        ) as salary_rank
FROM employees
ORDER BY salary
```

Grouping and group filtering

- `GROUP BY`: group rows with same values
- `HAVING`: filter groups based on aggregate conditions

```
SELECT column1, aggregate_function(column2)
FROM table_name
WHERE condition
GROUP BY column1
```

Grouping examples

ID	First Name	Last Name	Department	Salary	Hire Year	Job Level
1	John	Doe	Sales	$55,000	2022	Senior
2	Jane	Smith	IT	$65,000	2021	Senior
3	Mike	Johnson	Sales	$52,000	2023	Junior
4	Sarah	Wilson	HR	$48,000	2022	Senior
5	David	Brown	IT	$72,000	2020	Senior
6	Lisa	Davis	Marketing	$59,000	2023	Senior
7	Tom	Miller	Sales	$58,000	2021	Senior
8	Amy	Garcia	IT	$68,000	2022	Senior
9	Chris	Lee	Sales	$45,000	2023	Junior
10	Emma	Taylor	IT	$63,000	2023	Junior
11	Alex	Moore	Marketing	$51,000	2022	Junior
12	Ryan	White	HR	$46,000	2023	Junior

```sql
-- Count employees by department
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department

-- Multiple grouping columns
SELECT department, salary, COUNT(*) AS count
FROM employees
GROUP BY department, job_title

-- Departments with more than 5 employees
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 5

-- Combining WHERE and HAVING
SELECT department, AVG(salary) AS avg_salary
FROM employees
WHERE hire_date >= '2020-01-01'  -- Filter before grouping
GROUP BY department
HAVING AVG(salary) > 45000      -- Filter after grouping
```

Exercise 1

1. Download the Chinook_Sqlite database from https://github.com/lerocha/chinook-database/blob/master/ChinookDatabase/DataSources/Chinook_Sqlite.sqlite

   Look through its schema. There may be some lines you don't understand, you may ignore them for now.

2. From the Invoice table, find out how many orders are NOT from the USA.
3. From the Invoice table, find out how much each customer has spent.
4. From the Track table, find all tracks longer than 5 minutes.
5. From the Track table, rank tracks by length within each album (in descending order).

Multi-table relations

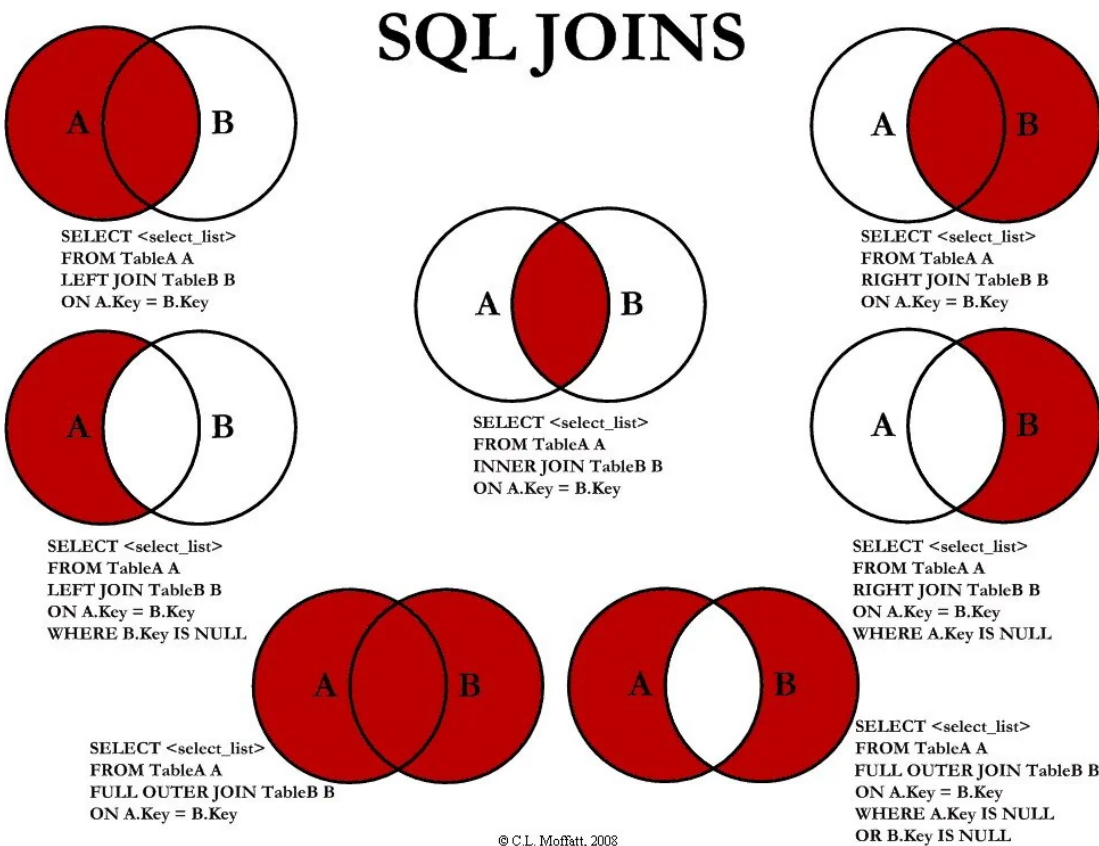
- Different tables in a database may be related
  - One-to-many
  - Many-to-one
  - Many-to-many

} Examples?

Multi-table Queries

Basic syntax:

```
SELECT column1, column2, …
FROM table1 alias1
[JOIN_TYPE] table2 alias2
ON join_condition
```



Examples of different kinds of JOIN

employees

employee_id	name	department_id	salary
1	John Doe	1	$55,000
2	Jane Smith	2	$65,000
3	Mike Johnson	1	$52,000
4	Sarah Wilson	3	$48,000
5	David Brown	2	$72,000
6	Lisa Davis	NULL	$59,000
7	Tom Miller	1	$58,000

departments

department_id	department_name	location	manager
1	Sales	New York	Alice Johnson
2	IT	San Francisco	Bob Wilson
3	HR	Chicago	Carol Smith
4	Marketing	Boston	Dan Brown

```sql
SELECT
    e.name,
    d.department_name,
    d.location
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id
```

Both Lisa Davis and Marketing department will not show up

Examples of different kinds of JOIN

employees

employee_id	name	department_id	salary
1	John Doe	1	$55,000
2	Jane Smith	2	$65,000
3	Mike Johnson	1	$52,000
4	Sarah Wilson	3	$48,000
5	David Brown	2	$72,000
6	Lisa Davis	NULL	$59,000
7	Tom Miller	1	$58,000

departments

department_id	department_name	location	manager
1	Sales	New York	Alice Johnson
2	IT	San Francisco	Bob Wilson
3	HR	Chicago	Carol Smith
4	Marketing	Boston	Dan Brown

```sql
SELECT
    e.name,
    d.department_name,
    d.location
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id
```

All employees will be shown, but Lisa's department will be NULL

Examples of different kinds of JOIN

employees

employee_id	name	department_id	salary
1	John Doe	1	$55,000
2	Jane Smith	2	$65,000
3	Mike Johnson	1	$52,000
4	Sarah Wilson	3	$48,000
5	David Brown	2	$72,000
6	Lisa Davis	NULL	$59,000
7	Tom Miller	1	$58,000

departments

department_id	department_name	location	manager
1	Sales	New York	Alice Johnson
2	IT	San Francisco	Bob Wilson
3	HR	Chicago	Carol Smith
4	Marketing	Boston	Dan Brown

```sql
SELECT
    e.name,
    d.department_name,
    d.location
FROM employees e
FULL OUTER JOIN departments d
ON e.department_id = d.department_id
```

name	salary	department_name	location
John Doe	$55,000	Sales	New York
Jane Smith	$65,000	IT	San Francisco
Mike Johnson	$52,000	Sales	New York
Sarah Wilson	$48,000	HR	Chicago
David Brown	$72,000	IT	San Francisco
Tom Miller	$58,000	Sales	New York
Lisa Davis	$59,000	NULL	NULL
NULL	NULL	Marketing	Boston

Logical order of SQL Statements

```
SELECT columns              -- 5. Finally, select the columns
FROM table1                 -- 1. Start here - get the base table
JOIN table2 ON ...          -- 2. Join with other tables
WHERE conditions            -- 3. Filter rows
GROUP BY columns            -- 4. Group rows together
HAVING conditions           -- 6. Filter groups (after grouping)
ORDER BY columns            -- 7. Sort the final results
LIMIT number                -- 8. Limit the number of rows returned
```

Question: what's wrong with this statement?

```
SELECT salary * 7.85 AS salary_in_HKD
WHERE salary_in_HKD > 1000000
```

Exercise 2

1. Show customer names along with their invoice totals. Include customers who haven't made any purchases.
2. Show customer purchase details: customer name, track name, artist name, and album title for all purchases. You will need multiple JOIN statements for this.
3. Show the number of customers and total sales by country, including only countries with more than or equal to 2 customers. Order by total sales in ascending order.

Subqueries

- Sometimes a query can get quite complex. We want to break it down into smaller, clearer, more logical steps.
- Scalar subqueries:

```
-- Find tracks longer than average
SELECT Name, Milliseconds
FROM Track
WHERE Milliseconds > (SELECT AVG(Milliseconds) FROM Track);
```

- List subqueries:

```
-- Find customers from countries with >10 customers
SELECT FirstName, LastName, Country
FROM Customer
WHERE Country IN (
    SELECT Country
    FROM Customer
    GROUP BY Country
    HAVING COUNT(*) > 10
);
```

Common Table Expressions (CTEs)

- Temporary named result set that exists only for the duration of your query

```
WITH cte_name AS (
    -- Your subquery here
    SELECT columns FROM tables WHERE conditions
)
SELECT columns FROM cte_name WHERE conditions;
```

- Can have multiple CTEs

Exercise 3

1. Using a CTE, find customers who have spent more than $40 in total. Show their name, country, and total spent.
2. Identify "power customers" and their favorite genres.
   A power customer is someone in the top 20% of spenders, and their favorite genre is the one they've purchased the most tracks from.

   Hint: You may need to define the following 3 CTEs

   CTE 1: Customer total spending (with ranking/percentile)
   CTE 2: Customer genre purchase counts
   CTE 3: Each customer's top genre

Creating a database

- To create an empty database in SQLite, enter this in the shell:

```
sqlite3 database_name.db
```

- To create a table inside a database:

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] table_name (
    column_name data_type [column_constraints],
    column_name data_type [column_constraints],
    ...
    [table_constraints]
);
```

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] table_name (
    column_name data_type [column_constraints],
    column_name data_type [column_constraints],
    ...
    [table_constraints]
);
```

- Common data types supported by SQLite:
  - TEXT: variable-length text
  - INTEGER
  - REAL: floats
  - NUMERIC: either int or float
  - DATE: YYYY-MM-DD
  - DATETIME: YYYY-MM-DD HH:MM:SS
  - TIMESTAMP: number of seconds since Unix epoch
  - BOOLEAN

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] table_name (
    column_name data_type [column_constraints],
    column_name data_type [column_constraints],
    ...
    [table_constraints]
);
```

- Common column constraint
  - PRIMARY KEY
    - specify the column to be the primary key, cannot be NULL
  - NOT NULL
  - UNIQUE
  - CHECK (condition)
    - e.g. CHECK (col_name > 0)

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] table_name (
    column_name data_type [column_constraints],
    column_name data_type [column_constraints],
    ...
    [table_constraints]
);
```

- Common table constraints
  - PRIMARY KEY (col_1, col_2, …)
    - Composite primary keys
    - Used typically for “junction tables” to handle many-to-many tables
  - FOREIGN KEY (local_column) REFERENCES parent_table (parent_column)
    - Specify “local_column” to serve as a foreign key
    - Ensure referential integrity
  - UNIQUE, NOT NULL, CHECK
    - Also works for multiple columns

Importing from a given csv file

Terminal

```
sqlite > .mode csv
sqlite > .headers on # if your csv file has headers
sqlite > .import file.csv table_name
```

Modifying a database

- Renaming tables

```
ALTER TABLE old_table_name RENAME TO new_table_name;
```

- Renaming columns

```
ALTER TABLE old_table_name RENAME COLUMN old_column TO new_column;
```

- Inserting new rows

```
INSERT INTO table_name (col1, col2, …) VALUES (val1, val2, …);
```

- Inserting new columns

```
ALTER TABLE table_name ADD COLUMN col_name data_type [constraints];
```

Modifying a database

- Updating rows

```
UPDATE table_name SET col1=val, col2=val2, ... WHERE condition;
```

- Deleting rows

```
DELETE FROM table_name WHERE condition;
```

If you want more practice…

- Leetcode has a lot of SQL questions commonly found in interviews